

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский государственный национальный исследовательский университет»

Физико-математический институт

**ОТЧЕТ
О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
на тему
ИНТЕЛЛЕКТУАЛЬНЫЕ АЛГОРИТМЫ АДАПТИВНОГО
УПРАВЛЕНИЯ СВЕТОФОРОВ ДЛ Я ОПТИМИЗАЦИИ
ДОРОЖНОГО ТРАФИКА**

Направление: 01.04.02 Прикладная математика и информатика

Выполнил:

магистр 2 курса

_____ **Ю. В. Санников**

Научный руководитель:

Директор института, доктор физико-
математических наук

_____ **М.А. Барулина**

Руководитель практики:

к. ф.-м.н., начальник Центра
прикладной математики и физики

_____ **Д. В. Макаров**

Срок прохождения практики с «09» января 2026 г. по «23» апреля 2026 г.

Пермь 2026

РЕФЕРАТ

Научно исследовательская работа 25 с., 21 рис., 16 источн., 1 прил.

НЕЙРОННЫЕ СЕТИ, СИМУЛЯЦИЯ ДОРОЖНОГО ДВИЖЕНИЯ С
ИСПОЛЬЗОВАНИЕМ SUMO, SUMO, УМНЫЙ СВЕТОФОР.

Данная работа посвящена развертыванию системы валидирования нейросетевых моделей и архитектур для адаптивного управления светофором с учетом автомобильного и пешеходного трафиков.

Во введении представлено обоснование актуальности темы регулировки Дорожного движения за счет умного светофора с использованием моделей сегментации видео.

Первая глава содержит анализ предметной области.

Во второй главе анализ BaseLine архитектуры.

В третьей главе содержатся описание методов для развертывания системы SUMO для симуляции трафика и представлены результаты симуляций на различных параметрах.

В заключении подведен итог проделанной работы и приведены перспективы развития.

Приложение А содержит ссылку на репозиторий с исходным кодом программы.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. Существующие решения проблемы трафика.....	6
1.1. Адаптивный светофор.....	6
1.2. Использование ML для детекции автомобилей в России	6
1.2.1. Видеодетектор автотранспортных средств ИНФОПРО и Видеодетектор транспорта РТК	7
1.2.2. НПО ИТС: Паук Трафик.....	8
1.3. Использование светофоров, управляемых ИИ, в странах Европы	9
2. Анализ архитектуры	10
2.1. Выбор архитектуры BaseLine.....	10
2.2. Ultralytics YOLO	10
2.3. Ограничения при выборе датасета	13
3. Симуляция дорожного движения с использованием SUMO.....	14
3.1. Методы инициализации дорожной сетки и трафика машин	14
3.2. Методы для регулирования дорожного движения.....	17
3.3. Методы для анализа дорожного движения.....	18
3.4. Результаты симуляции дорожного движения.....	18
ЗАКЛЮЧЕНИЕ	21
ГЛОССАРИЙ.....	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	23
Приложение А ссылка на репозиторий с кодом программы и результатами сегментации	25

ВВЕДЕНИЕ

Данная работа посвящена исследованию возможностей умных светофоров, основанных на использовании искусственного интеллекта, с целью повышения пропускной способности дорожного движения.

В условиях современного города, пробки, особенно в час-пик, приводят к огромным экономическим, временным, психологическим и иным задержкам. По статистике из-за этой проблемы в крупных мегаполисах теряется около 40% от времени необходимого на поездку [1]. Для стран Европы и США ущерб, производимый скоплением автомобилей, может составлять миллиарды долларов. Например, в США на 2023 ущерб экономике от автомобильных пробок составил \$70,4 млрд [2]. В России, как и в остальных странах мира проблема пробок так же актуальна. За последние десять лет число легковых автомобилей в России выросло на 8.7 млн. Появление новых развязок и полос движения решает проблему в краткосрочном периоде, но из-за долгих сроков на строительство, стоимости этих работ и постоянно растущего трафика данное решение нельзя назвать оптимальным. Более дешевым решением является модернизация светофоров – простых регуляторов движения, не требующих масштабных перестроек. Современные светофоры работают по трехступенчатой схеме: часы пик, дневное время, ночь, но в случае непредсказуемости потока, данная система становится малоэффективной.

Исходя из исследований ученых пермского политеха, сокращение фаз светофора способствует сокращению заторов на 15-20%, а значит своевременное переключение фаз светофоров будет способствовать более высокой пропускной способности [3]. Существующие адаптивные светофоры частично решают проблему трехступенчатой системы, но они часто работают на устаревших технологиях и сильно зависят от множества физических датчиков, чувствительных к атмосферным условиям, что снижает их эффективность.

Представленное в научно исследовательской работе решение заключается в разработке нейросетевых светофоров, которые будут более устойчивы к внешним влияниям и способны эффективно распознавать транспортные средства и пешеходов. Системы, основанные на искусственном интеллекте, могут самостоятельно анализировать дорожную ситуацию и оперативно переключать сигналы светофора в зависимости от загруженности. Это позволит минимизировать время ожидания на перекрестках и улучшить общую пропускную способность транспортных потоков.

В рамках предыдущей работы был изучен материал, необходимый для построения архитектуры модели, а также рассмотрены проекты решающие похожие задачи. Также был

разработан класс, использующий модель YOLO11l-seg, для сегментирования видео с целью определения каждого пикселя на принадлежность к транспортным средствам или людям. В заключении разработанный класс был протестирован на подготовленных заранее видеороликах.

В рамках текущей работы необходимо развернуть систему для симуляции дорожного движения SUMO и провести анализ полученных после множества симуляций данных.

Процесс развертывания можно разбить на несколько этапов:

Этап инициализации — написание скриптов для инициализации дорожной сетки и трафика машин.

Симуляция на различных параметрах — запуск симуляции с различным количеством машин и времени на переключение сигнала светофора.

Заключительная фаза — анализ полученных данных полученных после симуляции трафика.

Объектом исследования SUMO - бесплатный пакет программного обеспечения с открытым исходным кодом для моделирования транспортных потоков.

Предметом исследования являются api методы библиотеки traci, для управления дорожного движения и его анализа.

Целью научно исследовательской работы является развертывание системы валидирования нейросетевых моделей и архитектур для адаптивного управления светофором с учетом автомобильного и пешеходного трафиков.

Для достижения поставленной цели необходимо было решить следующие задачи:

1. Провести анализ существующих решений для адаптивного управления трафиком.
2. Выбрать BaseLine архитектуру.
3. Найти способы обхода ограничений для поиска датасета.
4. Развернуть систему для контроля трафика и его анализа.
5. Запустить и проанализировать симуляции с разными параметрами на обычном светофоре без модели.

Основная задача – рабочий алгоритм для запуска симуляции дорожного трафика с заданием параметров для управления продолжительности сигнала светофора.

Вспомогательная задача – алгоритм для сбора данных по пропускной способности каждого из светофоров.

Инструменты, использованные при разработке программы: Python, Jupiter Lab, SUMO.

1. Существующие решения проблемы трафика

Далее будут рассмотрены:

- адаптивный светофор,
- использование ML для детекции автомобилей в России,
- использование светофоров, управляемых ии, в странах Европы.

1.1. Адаптивный светофор

Адаптивные светофоры используют устаревшую технологию подсчета автомобилей на перекрестках, а именно физические датчики или видеодетектор фона в определенных зонах. Ограниченность этой технологии заключается в том, что обнаружение транспорта с помощью датчиков ограничено несколькими метрами перед переходом, а значит видеодетектор фона показывает заполненность только этого участка. К тому-же, если камера не сможет различить эту область на видео из-за расстояния, смещения или погодных условий, то светофор перестанет работать оптимально. Пример адаптивного светофора представлен на рисунке 1.

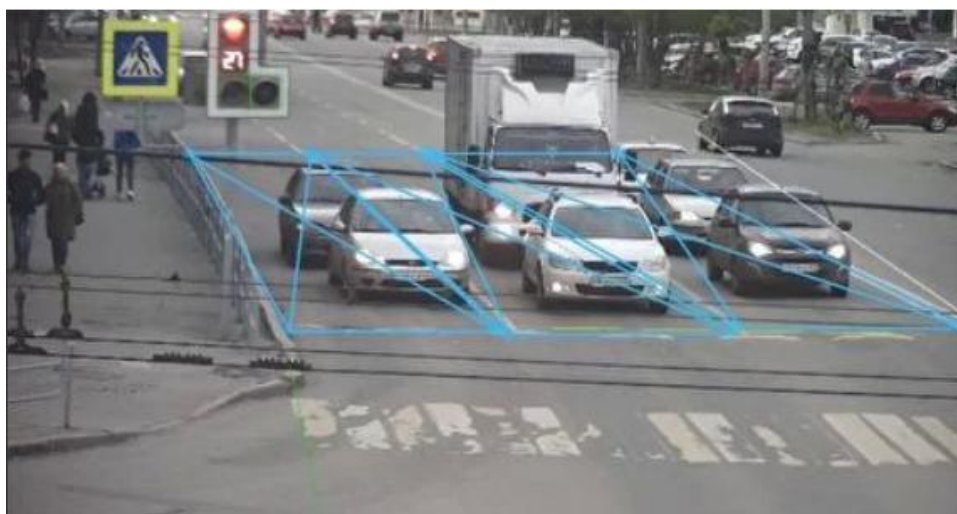


Рисунок 1. Видеодетектор адаптивного светофора.

Помимо устаревших технологий, установка физических датчиков для зоны детекции требует реконструкции дорожного полотна и местами стоимость установки может исчисляться миллионами [4 - 5].

Таким образом, адаптивные светофоры работают только в ситуациях, где правила переключения абсолютно очевидны или на мало загруженных перекрестках.

1.2. Использование ML для детекции автомобилей в России

Преимущества нейросетевых светофоров над адаптивными:

- не нуждаются в установке физических датчиков,

- получают более информативную информацию, например, тип транспорта, его длина,
- помимо транспортных средств, могут учитывать скопления пешеходов,
- количество объектов для детекции не ограничивается только типом транспорт и пешеход.

Рассмотренный в предыдущем пункте адаптивный светофор, хоть и умеет переключать сигналы светофора за счет датчиков, но делает это без использования компьютерного зрения. Детекция автомобилей в России используется в основном для фиксации нарушений на дорогах.

Далее будут рассмотрены проекты из России использующие компьютерное зрение для контроля движения на дорогах:

- видеодетектор автотранспортных средств ИНФОПРО и видеодетектор транспорта РТК,
- НПО ИТС: Паук Трафик,

1.2.1. Видеодетектор автотранспортных средств ИНФОПРО и Видеодетектор транспорта РТК

Оба проекта работают по принципу адаптивных светофоров, но используют нейросетевые алгоритмы для вычисления геометрического центра транспортного средства. Особенности этих проектов являются получение и анализ статистики по следующим параметрам: объем потока, средняя скорость, занятость дороги, интервал следования автотранспортных средств [6]. Система ИНФОПРО, может переключаться в режим адаптивного светофора [7], когда Видеодетектор РТК может похвастаться определением класса транспортного средства. На рисунке 2 представлен интерфейс Видеодетектора РТК, а на рисунке 3 видеодетектор ИНФОПРО.

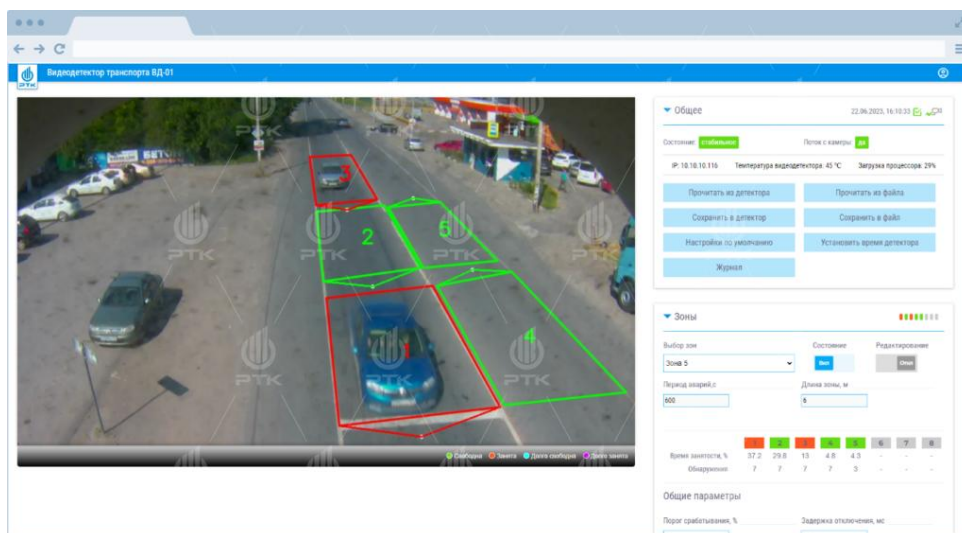


Рисунок 2. Видеодетектор РТК.

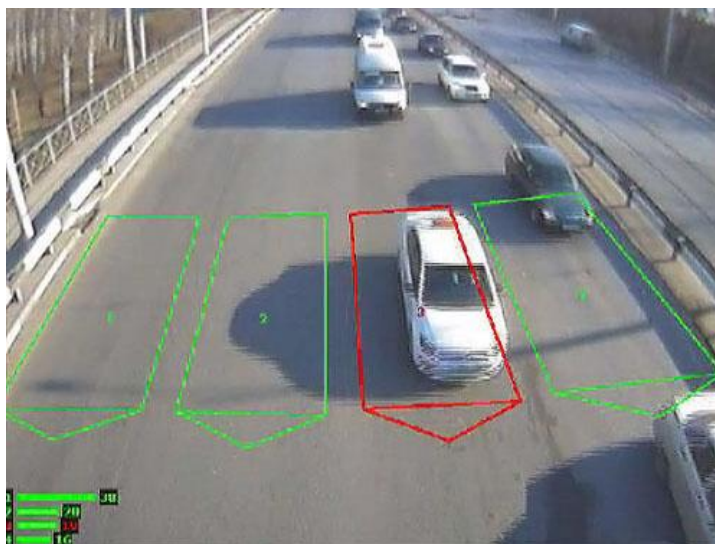


Рисунок 3. Видеодетектор ИНФОПРО.

1.2.2. НПО ИТС: Паук Трафик

Проект НПО ИТС: Паук Трафик тоже используется для аналитики дорожного движения, но в отличие от рассмотренных ранее проектов активно использует технологии компьютерного зрения. Видеодетектор способен определять количество автомобилей, плотность трафика, среднюю скорость на дороге и индивидуальную скорость транспортного средства, а также его тип. Разработанный инженерами «НПО «ИТС» детектор устойчив к погодным условиям и может контролировать движение даже на многополосных магистралях с погрешностью детектирования не выше 5%. Помимо анализа и сбора данных, комплекс видеофиксации «ПАУК» выявляет факты нарушения скоростного режима, проезда на запрещающий сигнал светофора, правил пользования ремнями безопасности и телефоном во время движения, пересечение стоп-линии и другие

нарушения ПДД, а также умеет распознавать госномера нарушителей [8]. На рисунке 4 представлен видеодетектор Паук Трафик.

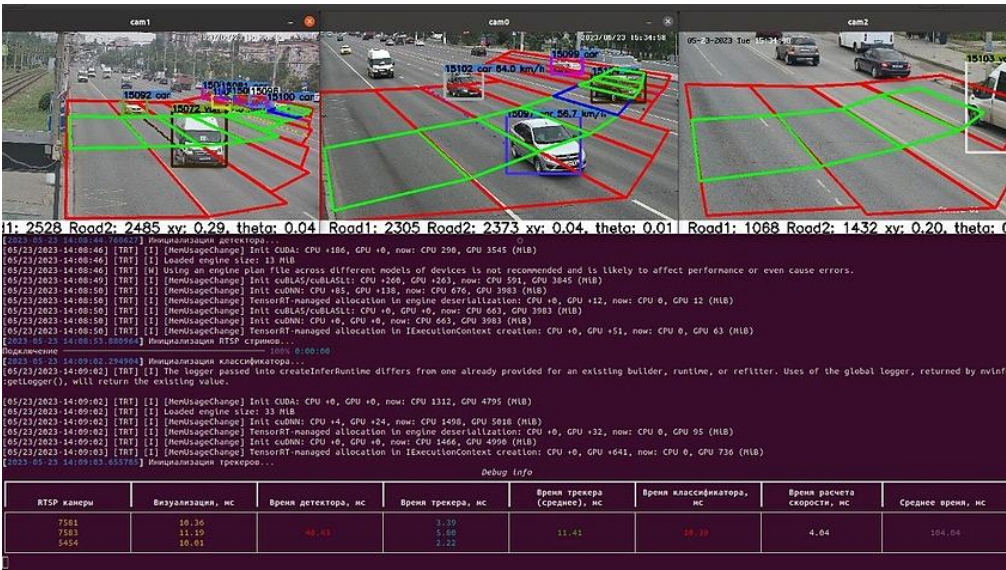


Рисунок 4. Видеодетектор Паук Трафик.

1.3. Использование светофоров, управляемых ИИ, в странах Европы

В отличие от, Российских проектов с использованием Компьютерного зрения, для анализа дорожного движения, в Германии уже проводятся испытания светофоров с искусственным интеллектом.

Так в 2023 году в немецком городе Лемго проект KI4LSA установил светофоры с камерами высокого разрешения и радарными датчиками, для фиксирования реальной дорожной ситуации, включая среднюю скорость автомобилей и время ожидания, для интеллектуального прогнозирующего переключения светофоров [9 - 10].

В конце наблюдений, использование искусственного интеллекта улучшило транспортный поток на 10-15% [11]. На рисунке 5 представлен видеодетектор KI4LSA.



Рисунок 5. Видеодетектор KI4LSA.

2. Анализ архитектуры

Далее будут рассмотрены:

- Выбор архитектуры BaseLine,
- Ultralytics YOLO,
- Ограничения для выбора датасета,

2.1. Выбор архитектуры BaseLine

Базовая архитектура для будет основана на архитектуре, разработанной в 2024 году старшими членами IEEE в рамках исследовательской работы «Edge ML Technique for Smart Traffic Management in Intelligent Transportation Systems» [12].

Архитектура, представленная на рисунке 6 состоит из 3 частей:

1. Сегментация кадров видеоряда.
2. Полученный на предыдущем шаге кадр обрабатывается с помощью обученной модели YOLO для сегментации объектов (машин).
3. Количество машин передается в алгоритм для переключения светофора.

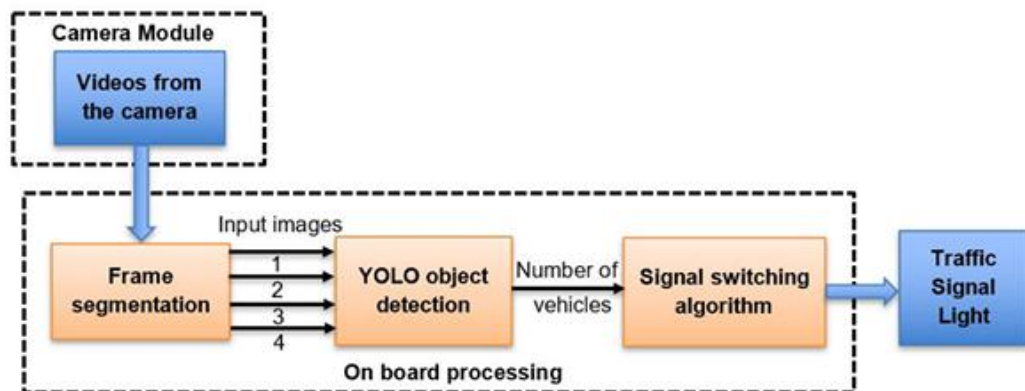


Рисунок 6. BaseLine архитектура.

В рамках представленного исследования для детекции автомобилей использовалась модель YOLOv3. Для детекции автомобилей в моей дипломной работе будет использоваться модель YOLOv11.

В рассматриваемой работе алгоритм используются три алгоритма для переключения сигналов светофора. В рамках предстоящей дипломной работы для регулирования продолжительности сигналов будет использоваться обученная модель.

2.2. Ultralytics YOLO

Задача сегментации – одна из ключевых задач в области компьютерного зрения, делится на три группы [13]:

– семантическая сегментация, тип сегментации, классифицирующий каждый пиксель изображения, без возможности определить разные экземпляры одного и того-же объекта (рисунок 12);



Рисунок 7. *Пример семантической сегментации.*

– instance segmentation, тип сегментации, способный различать не только класс объекта, но и различные экземпляры (рисунок 8);



Рисунок 8. *Пример instance сегментации.*

– паноптическая сегментация, распознает не только отдельные экземпляры объектов, но и фоновые (небо/земля), что позволяет более комплексно анализировать сцены (рисунок 9).

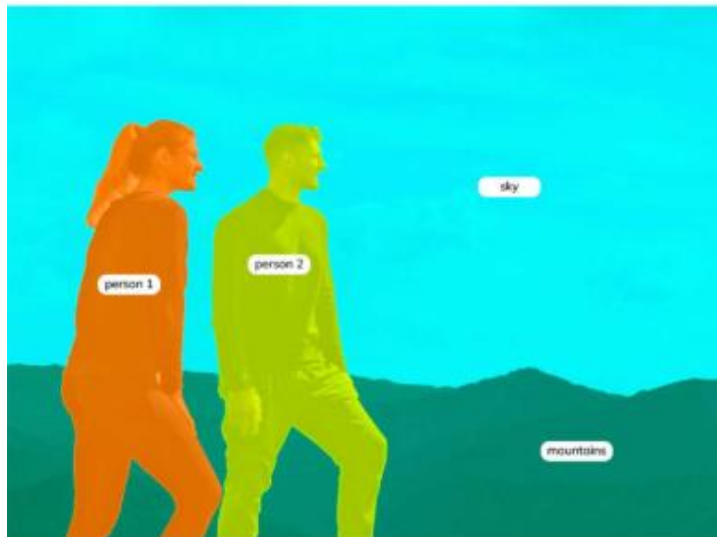


Рисунок 9. Пример паноптической сегментации.

В текущей работе, для определения объектов и их наличия будет использоваться метод Instance Segmentation.

Платформа ultralytics YOLO (You Only Look Once) специализируется на быстром и точном обнаружении объектов, а также поддерживает задачи сегментации. Помимо быстрого и точного определения объектов, система указывает количество экземпляров, обнаруженных на картинке, что необходимо для решения текущей задачи.

В рамках НИР использовалась YOLO11 с трехсекционной архитектурой Backbone, Neck и Head [14], представленной на рисунке 10.

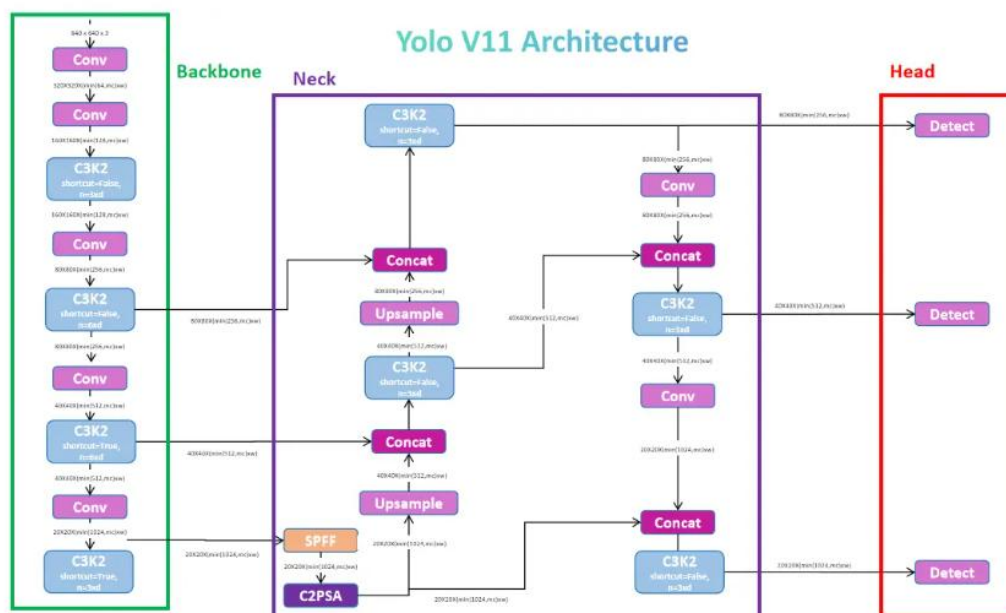


Рисунок 10. Архитектура YOLO v11.

Backbone часть извлекает из изображения полезные признаки на основе эффективного “bottleneck-based” алгоритма.

Neck – обрабатывает выходные данные Backbone и передает полученные признаки головной части

Head (головная) часть, работает исходя из поставленной задачи:

- обнаружение объектов на изображении,
- семантическая сегментация объектов,
- определение ключевых точек для позы (например, на человеке голова, локти и т.д.),
- классификация изображений и т.д.

Bottleneck-based слой представляет из себя два сверточных блока, соединенных последовательно с функцией конкатенации, если параметр `shortcut = true`, входные параметры объединяются с выходными параметрами второго сверточного слоя. Такая структура повышает эффективность и процесс обучения. На рисунке 16 представлен Bottleneck слой.

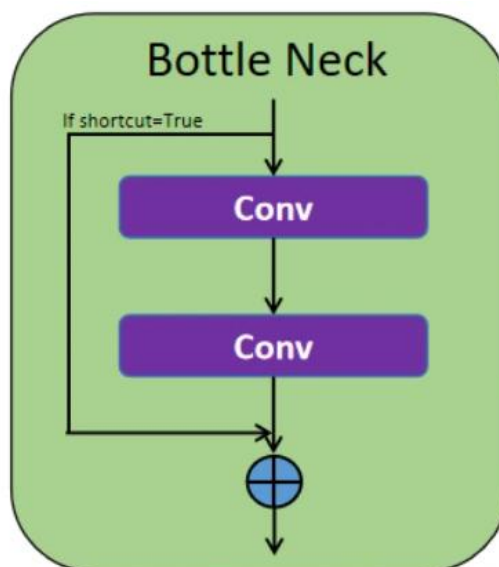


Рисунок 11. Bottleneck слой.

2.3. Ограничения при выборе датасета

Во время поиска датасетов для обучения модели YOLO всплыли две проблемы:

1. на популярных сайтах с датасетами, не получилось найти подходящих размеченных данных,
2. чтобы получить записи с муниципальных камер наблюдения необходимо обращаться в правоохранительные органы.

Для решения проблем можно использовать любые датасеты с размеченными машинами, но дополненные собственными размеченными данными с онлайн трансляций перекрестков дорог.

3. Симуляция дорожного движения с использованием SUMO

SUMO — это открытый, высокопортативный пакет симуляции трафика, предназначенный для работы с крупными сетями. Он позволяет проводить симуляцию, объединяющую несколько видов транспорта (легковой, грузовой и т.д.), включая пешеходов, и оснащается большим набором инструментов для создания сценариев [15]. Для модификации объектов дорожной сети или получения информации о них используется библиотека `traci`. `Traci` (Traffic Control Interface) – это API-модуль, предоставляющий возможность взаимодействия с симулятором движения SUMO в режиме реального времени [16].

Будут рассмотрены:

- методы инициализации дорожной сетки и трафика машин,
- методы для регулирования дорожного движения,
- методы для анализа дорожного движения,
- результаты симуляции дорожного движения.

В качестве результатов будут представлены результаты метрик и их сравнение. По результатам сегментации будут выявлены недочеты использования обычного светофора.

3.1. Методы инициализации дорожной сетки и трафика машин

Для быстрой инициализации сети дорог был разработан метод `initialize`, в метод передается размерность сетки, ширина полос, и длина дорог. На рисунке 12 вызывается команда с помощью `os.system` – в консоли системы вызывается метод `netgenerate` со следующими параметрами:

- `--grid` (создать сетку)
- `--grid.number=` (размер сетки)
- `-L=` (количество полос на каждой из дорог)
- `--tls.guess` (задать шаблон для пересечений дорог)
- `--grid.length` (длина каждой из дорог сетки)
- `--output-file=` (выходной файл сети, записанный в xml формат)

```
def initialize(grid=3,lanes=2,length=200):  
    os.system("netgenerate --grid --grid.number="+str(grid)+" -L="+str(lanes)+" --tls.guess  
--grid.length="+str(length)+" --output-file=grid.net.xml")
```

Рисунок 12. Метод для инициализации дорожной сети.

Далее будет рассмотрен метод для задания остальных параметров симуляции `single`. В метод симуляции передается количество машин в симуляции (`vehicles`), путь до папки

SUMO на компьютере, в которой хранятся скрипты для быстрого развертывания сети (*path*), папка для записи свойств каждой из машины в симуляции, а также результатов самой симуляции (*gridsDataFolder*), параметр *showVisualization* отвечает за включение визуализации симуляции во время прогона, *green_light_duration* и *red_light_duration* – продолжительности зеленого и красного сигналов соответственно. На рисунке 13 представлены команды для генерации потоков (используется скрипт из папки инструментов SUMO – *randomTrips.py*), транспортных маршрутов, вероятности поворотов на перекрестках (вероятность что автомобиль поедет в выбранном направлении, используется скрипт из папки инструментов SUMO – *generateContinuousRerouters.py*) соответственно.

```
def single(vehicles, path, gridsDataFolder, showVisualization = False,
green_light_duration = 30, red_light_duration = 30):
    sumoRunCfg = 'sumo-gui' if showVisualization else 'sumo'
    os.system('python "'+path+'randomTrips.py" -n grid.net.xml -o flows.xml --begin 0 --end
1 --period 1 --flows '+str(vehicles))
    os.system("jtrrouter --flow-files=flows.xml --net-file=grid.net.xml --output-
file=grid.rou.xml --begin 0 --end 10000 --accept-all-destinations")
    os.system('python "'+path+'generateContinuousRerouters.py" -n grid.net.xml --end 10000 -
o rerouter.add.xml')
```

Рисунок 13. Инициализация поведения на сетке дорог.

Для инициализации самих светофоров и детекторов, для подсчета автомобилей используется метод *create_additional_with_e1_detectors*, который принимает на вход путь до файла с сетью дорог «*grid.net.xml*» и путь куда будет сохраняться информация во время симуляции для каждого из детектора. Суть метода заключается в том, что перед каждой дорогой на пересечении (*junction*) устанавливаются датчики, считывающие движение, информация по каждой секунде итерации записывается в файл *detector_{id_перекрестка}_{номер_полосы}.xml*. На рисунке 14 представлена часть алгоритма, отвечающая за описанное выше поведение.

```

# Находим все junction и их входящие lane
for junction in root.findall("junction"):
    junction_id = junction.get("id")
    if not junction_id:
        continue

    # Ищем все connection, ведущие к этому junction
    connections = root.findall(f"../connection[@tl='{junction_id}']")
    lanes = set()
    for conn in connections:
        edge_id = conn.get("from")
        lane_idx = conn.get("fromLane")
        if edge_id and lane_idx:
            lanes.add(f"{edge_id}_{lane_idx}")

    # Создаем детектор E1 для каждой полосы
    for i, lane_id in enumerate(lanes):
        detector = ET.SubElement(additional_root, "inductionLoop")
        detector.set("id", f"{junction_id}_E1_{i}")
        detector.set("lane", lane_id)
        detector.set("pos", "-10")
        detector.set("period", "1")
        os.makedirs(path_to_save_data, exist_ok=True)
        detector.set("file", f"{path_to_save_data}/detector_{junction_id}_{i}.xml")

```

Рисунок 14. Инициализация детекторов для каждой полосы перед светофором.

На рисунке 15 представлен результат полной инициализации сетки.

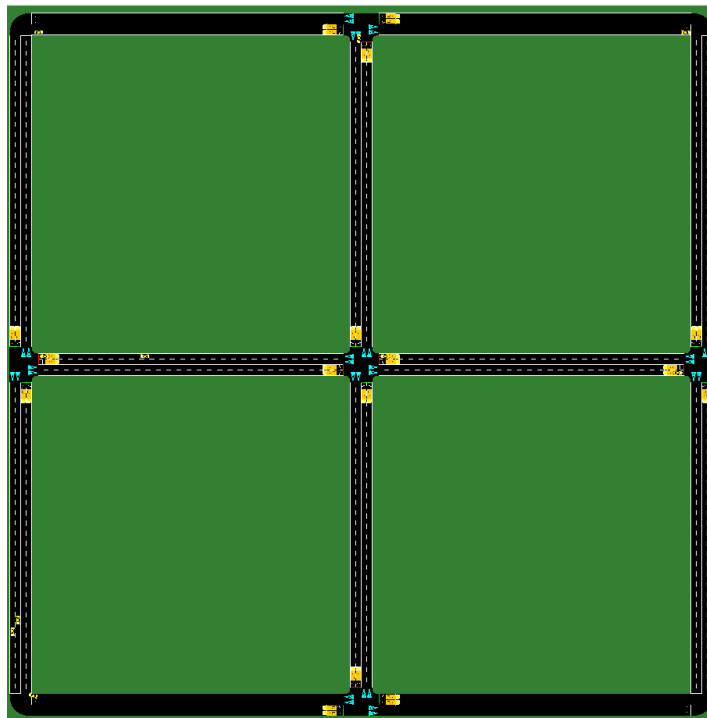


Рисунок 15. Инициализированная сетка 3x3 с рабочим светофором (длительность сигнала 20 сек) и 10 машинами в симуляции.

3.2. Методы для регулирования дорожного движения

Для установки продолжительности сигналов светофора был написан метод `setTrafficLightFazes`, представленный на рисунке 16. Сначала заполняется массив с фазами в следующем порядке: зеленый -> желтый, желтый -> красный, красный -> желтый, желтый -> зеленый. С помощью api метода `setCompleteRedYellowGreenDefinition` устанавливаются новые значения для каждой из фаз.

```
def setTrafficLightFazes(traci, id, green_light_duration, red_light_duration):
    phases = []
    # Установка продолжительности зеленого сигнала
    phases.append(traci.trafficlight.Phase(green_light_duration, "GGgrrrrrGGgrrrrr", 0, 0))
    # Установка продолжительности переключения на красный сигнал
    phases.append(traci.trafficlight.Phase(3, "yyyrrrrryyyrrrrr", 0, 0))
    # Установка продолжительности красного сигнала
    phases.append(traci.trafficlight.Phase(red_light_duration, "rrrrGGgrrrrrGGgr", 0, 0))
    # Установка продолжительности переключения на зеленый сигнал
    phases.append(traci.trafficlight.Phase(0, "rrrryyyrrrrryyyr", 0, 0))
    logic = traci.trafficlight.Logic("0", 0, 0, phases)
    traci.trafficlight.setCompleteRedYellowGreenDefinition(id, logic)
    detector.set("file", f"{path_to_save_data}/detector_{junction_id}_{i}.xml")
```

Рисунок 16. Метод для установки продолжительности сигналов светофора.

Далее в самой симуляции на каждом шаге, с помощью метода `get_all_vehiclesCount_for_traffic_light` вычисляется количество машин, проехавших датчики, ведущие к развязке `tId`. В конце симуляции данные записываются в csv файл. На рисунке 17 представлен алгоритм для описанного выше поведения.

```
...
# симуляция длится 10000 секунд
while traci.simulation.getTime() < 10000:
    traci.simulationStep()
    # print(traci.simulation.getTime())
    for tId in allTrafficLightsIds:
        all_sums[tId] += get_all_vehiclesCount_for_traffic_light(tId,
allInductionLoops)

all_sums_csv = {k: [v] for k, v in all_sums.items()}
print("passed trough: ",all_sums_csv)
all_sums_path = dataFolder + analysis.getCSVFileName(vehicles)
saveToCSV(data=all_sums_csv, path=all_sums_path)
...
```

Рисунок 17. Часть кода, отвечающая за саму симуляцию.

На рисунке 18 представлен метод `get_all_vehiclesCount_for_traffic_light`. Его суть заключается в том, что находятся все датчики, расположенные около светофора с `id`

trafficLightId. Далее с помощью api метода *traci.inductionloop.getLastIntervalVehicleNumber* запрашивается значение о количестве проехавших через датчик машин и складывается к общей сумме проехавших машин у выбранного светофора.

```
def get_all_vehiclesCount_for_traffic_light(trafficLightId, allInductionLoopIds):
    curSum = 0
    trafficLightIndicators = [item for item in allInductionLoopIds if trafficLightId in
item]
    for detector_id in trafficLightIndicators:
        curSum +=traci.inductionloop.getLastIntervalVehicleNumber(detector_id)
    return curSum
```

Рисунок 18. Метод для подсчета проехавших через светофор *trafficLightId* машин.

3.3. Методы для анализа дорожного движения

Для анализа прогнанных симуляций был разработан метод *plots_mean_traffic*, считывающий данные csv файлов, в которые были записаны количества машин, которые проехали через каждый из светофоров и выводящий среднее значение для каждой из симуляции.

Часть метода представлена на рисунке 19.

```
for duration in durations:
    duration_mean_array = []
    for i in range(0,len(vehicles)):
        cur_count = vehicles[i]
        data_folder = get_dataFolder(pathToSave, cur_count, green_faze=duration,
red_faze=duration)
        df = pd.read_csv(data_folder + getCSVFileName(cur_count))
        duration_mean_array.append(df.iloc[0].mean())
    new_array = np.array(duration_mean_array)
    dfAll = pd.concat([dfAll, pd.DataFrame([np.insert(new_array, 0, duration)],
columns=dfAll.columns)], ignore_index=True)
```

Рисунок 19. Часть метода, отвечающая за сбор информации из csv файлов и записи в *pandas DataFrame*.

Также были написаны метод для подсчета отношения плотности трафика к потоку машин и метод для подсчета отношения скорости машины к плотности трафика (*plots*).

3.4. Результаты симуляции дорожного движения

На рисунке 20 представлен график демонстрирующий среднее количество проехавших через светофор машин для различной продолжительности фазы светофора. Различные линии обозначают количество машин, запущенных в симуляцию.

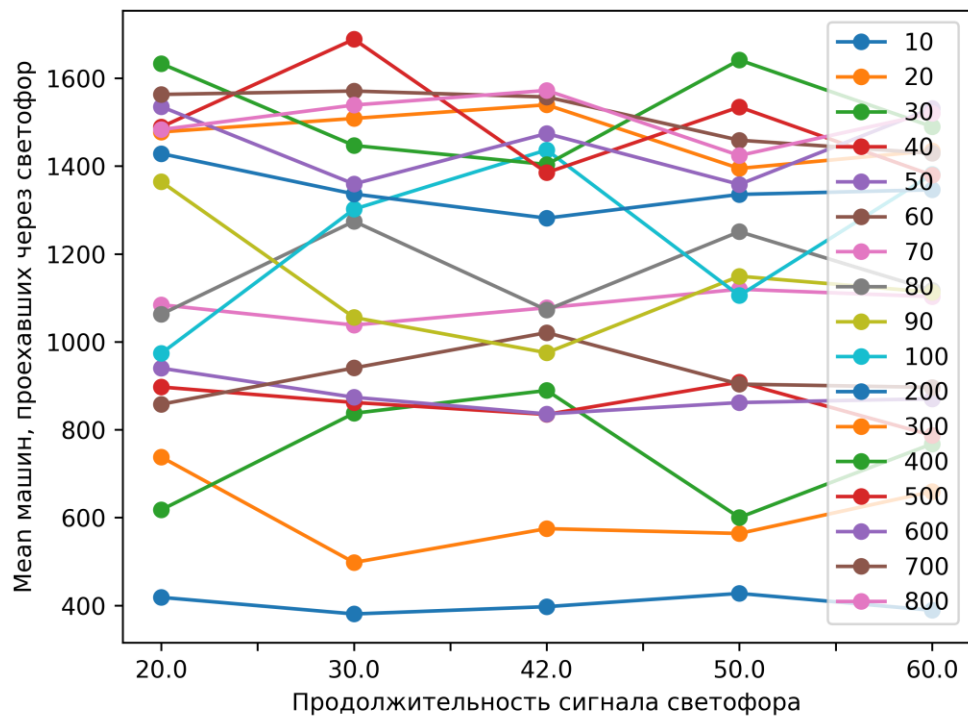


Рисунок 20. График демонстрирующий среднее количество проехавших через светофор машин для различной продолжительности фазы светофора.

По результатам симуляций можно сказать, что увеличение продолжительности фаз светофора не влияет на количество машин, проехавших светофор.

На рисунке 21 представлены графики демонстрирующий отношение скорости машины к загруженности трафика для фаз светофора 20 секунд слева и 60 секунд справа.

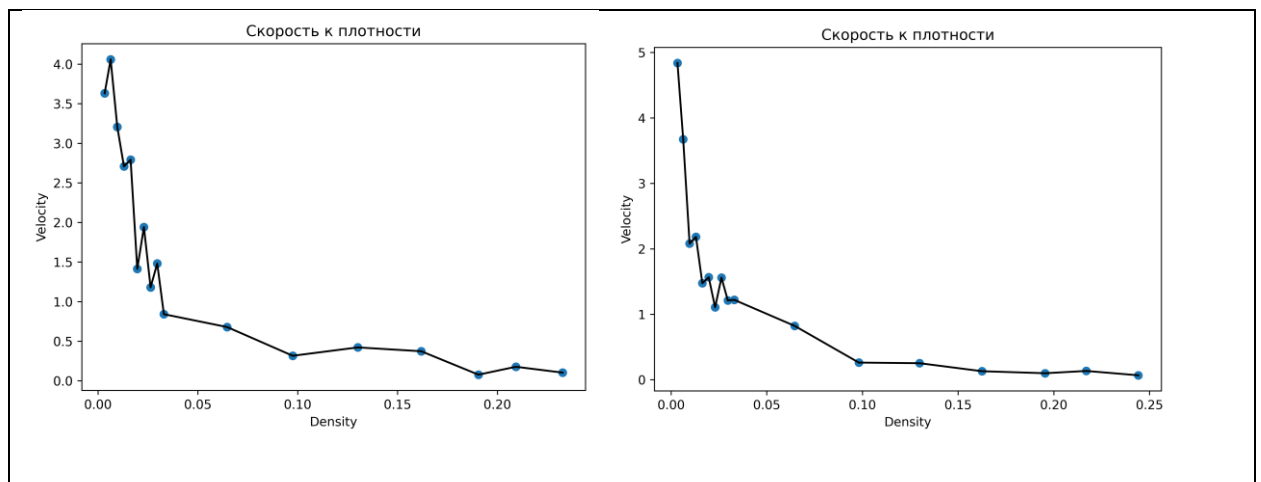


Рисунок 21. Часть метода, отвечающая за сбор информации из csv файлов и записи в pandas DataFrame.

По графикам можно определить, что увеличение продолжительности фаз светофора не сильно влияет на среднюю скорость движения автомобиля.

По результатам анализа, можно сделать вывод, что изменение фаз светофора не влияет на общую пропускную способность или скорость, чтобы улучшить результат нужно запускать симуляцию для каждого параметра несколько раз и брать среднее значение.

ЗАКЛЮЧЕНИЕ

В результате выполнения научно исследовательской работы был изучен материал, необходимый для развертывания системы валидирования нейросетевых моделей и архитектур для адаптивного управления светофором с учетом автомобильного и пешеходного трафиков. В заключении на развернутой системе были запущены симуляции и проанализированы.

Были выполнены задачи, а именно:

- Выделены ограничения датасетов и их возможные обходы,
- Выбран BaseLine для Дипломной работы,
- Развернута система для анализа трафика,
- Проведены тестовые запуски системы с разными параметрами и представлены результаты симуляций.

Таким образом, цель и поставленные задачи были достигнуты.

Для дальнейшего развития необходимо:

- адаптировать систему под обученную модель, для переключения сигналов,
- добавить в симуляцию разные типы транспорта, пешеходов и ситуации,
- реализовать 3д визуализацию.

ГЛОССАРИЙ

BaseLine (базовая) архитектура —простая, начальная структура системы (в ML, ПО или ИТ-инфраструктуре), определяющая минимально приемлемый уровень качества или производительности.

SUMO (Simulation of Urban MObility) - бесплатный пакет программного обеспечения с открытым исходным кодом для моделирования транспортных потоков.

Senior Member, IEEE (Старший член IEEE) - профессиональный статус, присваиваемый Институтом инженеров электротехники и электроники (IEEE) специалистам, имеющим значительные достижения, опыт работы не менее 10 лет и рекомендации от коллег.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 А.Б. Рахманов. Четыре колеса апокалипсиса: причины автомобильных пробок в крупных странах мира [Научная статья] // cyberlinka.ru [Сайт]. URL: <https://cyberleninka.ru/article/n/chetyre-kolesa-apokalipsisa-prichiny-avtomobilnyh-probok-v-krupnyh-gorodah-mira/viewer> (Дата последнего обращения 22.12.2025)
- 2 В США подсчитали, во сколько обходятся стране автомобильные пробки [Электронный ресурс] // officelife.media [Сайт]. URL: <https://officelife.media/news/54320-v-ssha-podschitali-vo-skolko-obkhodyatsya-strane-avtomobilnye-probki-summa-vpechatlyaet/> (Дата последнего обращения 22.12.2025)
- 3 Ученые Пермского Политеха разработали алгоритм оптимизации перекрестков на основе компьютерного моделирования [Электронный ресурс] // pstu.ru [Сайт]. URL: <https://pstu.ru/news/2025/10/17/17823/> (Дата последнего обращения 22.12.2025)
- 4 Как сделать светофор умнее, а проезжать и загруженные перекрестки быстрее? [Электронный ресурс] // habr.com [Сайт]. URL: <https://habr.com/ru/articles/553162/> (дата последнего обращения 22.12.2025) .
- 5 На «умные» светофоры в Уфе в 2023 году потратили более 270 млн рублей [Электронный ресурс] // www.kommersant.ru [Сайт]. URL: <https://www.kommersant.ru/doc/6509735> (дата последнего обращения 22.12.2025).
- 6 Видеодетектор автотранспортных средств ИНФОПРО [Электронный ресурс] // infoprocess.ru [Сайт]. URL: <https://www.infoprocess.ru/products.html> (дата последнего обращения 22.12.2025).
- 7 Видеодетектор автотранспортных средств ИНФОПРО сценарии использования [документация] <https://www.infoprocess.ru/downloads/scenario.pdf> (дата последнего обращения 22.12.2025).
- 8 НПО ИТС: Паук Трафик [электронный ресурс] // www.tadviser.ru [Сайт] URL: <https://clck.ru/3QxVQh> (дата последнего обращения 22.12.2025).
- 9 В Германии тестируют умные светофоры Ими управляют самообучающиеся алгоритмы [электронный ресурс] // digitalocean.ru [Сайт] URL: <https://digitalocean.ru/n/v-germanii-testiruyut-umnye-svetofory> (дата последнего обращения 22.12.2025).
- 10 Светофоры с искусственным интеллектом начали испытывать в Германии [электронный ресурс] // kolesa.kz [Сайт] URL: <https://kolesa.kz/content/news/svetofory-s-iskusstvennym-intellektom-nachali-ispytyvat-v-germanii/> (дата последнего обращения 22.12.2025).

- 11 KI4LSA Künstliche Intelligenz für Lichtsignalanlagen. (KI4LSA Искусственный интеллект для Системы световой сигнализации) [научная статья] URL: https://www.iosb-ina.fraunhofer.de/content/dam/iosb/iosb-ina/documents/WhitePaper_KI4LSA.pdf (дата последнего обращения 22.12.2025).
- 12 ANAKHI HAZARIKA, MOUSTAFA M. NASRALLA, NIKUMANI CHOUDHUR, SOHAIB BIN ALTAF KHATTAK, IKRAM UR REHMAN. Edge ML Technique for Smart Traffic Management in Intelligent Transportation Systems [Научная статья] // arxiv.org [Сайт]. URL: <https://drive.google.com/file/d/1zYmKNS5f-iOmcibpucmLvaoQdne4uqS0/view?usp=sharing> дата последнего обращения 27.04.2026).
- 13 Семантическая сегментация: самый полный гайд 2024. [Электронный ресурс] // habr.com [Сайт]. URL: https://habr.com/ru/companies/data_light/articles/855336/ (дата последнего обращения 22.12.2025).
- 14 Nikhileswara Rao Sulake Guide on YOLOv11 Model Building from Scratch using PyTorch. [документация] // www.analyticsvidhya.com [Сайт]. URL: <https://www.analyticsvidhya.com/blog/2025/01/yolov11-model-building/> (дата последнего обращения 22.12.2025).
- 15 SUMO User Documentation [документация] // sumo.dlr.de [Сайт]. URL: <https://sumo.dlr.de/docs/index.html> (дата последнего обращения 27.04.2026).
- 16 TraCI [документация] // sumo.dlr.de [Сайт]. URL: <https://sumo.dlr.de/docs/TraCI/index.html> (дата последнего обращения 27.04.2026).

Приложение А

ссылка на репозиторий с кодом программы и результатами сегментации

Репозиторий проекта можно найти, перейдя по ссылке:

https://github.com/Uranus28/gitDiploma_temp (github.com)